



SGCP's
Guru Nanak Khalsa College
of Arts, Science & Commerce (Autonomous)



NGS - QUALITY CHECK TOOLKIT

MINI-PROJECT REPORT

DATE :
9th March, 2025

PREPARED BY :
Prarthi Kothari (115)
M. Sc. Bioinformatics (Part 2)

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have supported me throughout the journey of my research project titled ‘NGS – Quality Check Toolkit: A Comprehensive User – Friendly GUI for the Quality Check of Next – Generation Sequencing (NGS) Reads’.

Firstly, I extend my sincere thanks to Dr. Ratna Sharma (The Principal, of Guru Nanak Khalsa College of Arts, Science & Commerce (Autonomous)) and Dr. Gursimran Kaur Uppal (Head of Department of Bioinformatics), for fostering an environment that encourages exploration and creativity. Your leadership and vision have created a space where students can thrive and pursue their academic interests with passion.

Next, I would like to acknowledge the unwavering support of my guide, Prof. Sermarani Nadar, whose invaluable guidance, expertise, and encouragement have been instrumental in shaping this project. Your insights into the complexities of the field have not only enriched my understanding but have also inspired me to explore innovative solutions. Your dedication to teaching and your willingness to share your knowledge have provided me with a solid foundation in both theoretical and practical aspects of this research. I would also like to extend my heartfelt gratitude to Prof. Aparna Patil Kose, whose knowledge and support were instrumental in my academic growth.

I am also grateful to my peers and colleagues who contributed their insights and feedback during various stages of this project. The collaborative spirit within our department has made this research experience enriching and enjoyable.

Lastly, I would like to thank my family for their unwavering support and encouragement throughout this journey. Your belief in my abilities has been a constant source of motivation. This project is a culmination of collective efforts, and I am deeply appreciative of everyone who played a role in its progress.

INDEX

Chapter No.	Title	Page No.
1	Abstract	1
2	Objectives	3
3	Introduction	5
	I. Next – Generation Sequencing (NGS)	6
	II. Graphical User Interface (GUI)	8
4	Review of Literature	10
	Next – Generation Sequencing (NGS) Tools	11
	I. SRA Toolkit	12
	II. FastQC	13
	III. FastP	16
5	Methodology	18
6	Code	22
7	Results	35
9	Conclusion	46
10	References	48

CHAPTER 1: ABSTRACT

ABSTRACT

This report introduces the ‘NGS – Quality Check Toolkit’, a solution designed to streamline next-generation sequencing (NGS) workflows for checking the quality of the retrieved sequences. The toolkit simplifies complex NGS processes, making them accessible to researchers and clinicians with varying levels of expertise through an intuitive graphical user interface (GUI) for efficient data management. Key objectives include integrating essential NGS pipeline steps such as data acquisition and quality control into a cohesive platform. The toolkit incorporates bioinformatics tools like SRA Toolkit for data retrieval and FastQC and Fastp for quality assessment. The NGS pipeline emphasizes efficiency and accuracy, with automated processes to minimize user input and maximize throughput. In conclusion, the toolkit offers a robust and a user-friendly solution.

CHAPTER 2: OBJECTIVES

OBJECTIVES

1. Develop a User-Friendly GUI for NGS Quality Check Workflow

The goal is to design an intuitive GUI that simplifies NGS workflows for quality check, enabling researchers and clinicians to easily navigate and manage processes without technical expertise. Key features include drag-and-drop functionality and accessibility across devices. The focus is on enhancing user experience and efficiency in handling NGS tasks.

2. Integrate Key NGS Workflow Steps into the GUI

This objective involves integrating all major NGS workflow steps into the GUI, including data acquisition and quality control. Users can monitor progress and access real-time results, centralizing the workflow in one platform to reduce software switching and minimize errors.

3. Enable NGS Workflow Customization

The GUI will allow users to customize their NGS workflows based on specific project needs, enabling modifications like adding/removing steps and adjusting parameters. This flexibility supports diverse experimental designs.

4. Standardization and Quality Control Tools

As bioinformatics tools proliferate, ensuring the quality and reproducibility of analyses becomes crucial. There is a need for standardized tools that can assess the quality of data and methods used in bioinformatics studies, helping to maintain high research standards.

Building a user interface for NGS workflows for quality check is vital for enhancing user experience, streamlining data management and facilitating collaboration. By focusing on usability and accessibility, such interfaces can empower healthcare professionals to harness the full potential of next-generation sequencing, ultimately leading to advancements in the field.

CHAPTER 3: INTRODUCTION

INTRODUCTION

I. Next-Generation Sequencing (NGS)

Next-Generation Sequencing (NGS) is a groundbreaking technology that revolutionizes the field of genomics by enabling the rapid sequencing of DNA and RNA. Unlike traditional sequencing methods, NGS employs a massively parallel approach, allowing for the simultaneous determination of the nucleotide order in entire genomes or targeted regions. This capability results in ultra-high throughput and scalability, making it a powerful tool for both research and clinical applications.

Importance of NGS Workflows

1. Efficiency:

NGS workflows are designed to optimize the sequencing process, significantly enhancing the speed of sample preparation and analysis. This efficiency is vital in both research settings, where timely results can drive further investigations, and clinical environments, where rapid diagnostics can influence patient care decisions. By minimizing manual steps and automating processes, NGS workflows facilitate quicker turnaround times for obtaining genomic data.

2. Cost-Effectiveness:

The evolution of NGS has led to a dramatic reduction in sequencing costs over the past decade. As technologies have advanced, the price per base of DNA sequenced has decreased significantly, making high-throughput sequencing accessible to a broader range of researchers and institutions. This affordability empowers smaller labs and universities to engage in cutting-edge genomic research that was previously limited to well-funded facilities.



3. Data Management:

The vast amounts of data generated by NGS pose significant challenges in terms of management, analysis, and interpretation. Effective workflows incorporate robust data management systems that facilitate the handling of large datasets. These systems enable researchers to efficiently store, retrieve, and analyze genomic information, ensuring that meaningful insights can be derived from complex data sets without overwhelming computational resources.

4. Customization:

One of the key advantages of NGS workflows is their flexibility. Researchers can tailor their sequencing approaches based on specific experimental needs, whether they are focusing on whole-genome sequencing, targeted resequencing, or RNA sequencing. This customization allows for the optimization of protocols to suit different sample types or research questions, enhancing the relevance and applicability of the results obtained.

5. Advancements in Precision Medicine:

NGS workflows play a crucial role in advancing precision medicine by enabling the identification of genetic variants that inform treatment decisions. By analyzing an individual's genomic information, healthcare providers can tailor therapies to improve patient outcomes based on specific genetic markers associated with diseases. For instance, NGS can uncover mutations linked to cancer susceptibility or drug resistance, guiding oncologists in selecting personalized treatment regimens that enhance efficacy while minimizing adverse effects.

In summary, Next-Generation Sequencing (NGS) represents a transformative advancement in genomics that offers unparalleled efficiency and cost-effectiveness while paving the way for significant strides in precision medicine. As workflows continue to improve and adapt to emerging challenges, NGS will undoubtedly play a pivotal role in shaping the future of genetic research and personalized healthcare.

II. Graphical User Interface (GUI)

A Graphical User Interface (GUI) is a sophisticated type of user interface that enables users to interact with electronic devices through visual elements, such as graphical icons, buttons, and visual indicators. Unlike command-line interfaces that require text-based commands, GUIs provide a more intuitive and user-friendly experience, making technology accessible to a broader audience. GUIs are prevalent in software applications across various platforms, including desktop computers, mobile devices, and web applications, enhancing usability and overall efficiency.

Key Features of GUIs

1. Visual Elements:

GUIs utilize icons, buttons, menus, and windows to represent functions and data visually. This design allows users to navigate software applications more easily by clicking or tapping on graphical elements rather than typing commands.

2. Secondary Notation:

Secondary notation refers to additional visual cues that enhance understanding and usability. For instance, color coding or tooltips can provide contextual information about an icon's function, improving the user experience.

3. Interactive Components:

GUIs often include interactive components such as sliders, checkboxes, and dropdown menus that allow users to manipulate data and settings dynamically. This interactivity fosters a more engaging user experience.



Benefits of Implementing a GUI in NGS Workflows

1. Enhanced Efficiency:

By providing a visually intuitive interface, GUIs streamline the workflow for Next-Generation Sequencing (NGS) processes. Users can quickly access tools and functionalities without needing extensive training or familiarity with complex commands. This efficiency is particularly beneficial in high-throughput environments where time is critical.

2. Improved Data Management:

GUIs facilitate better data management by allowing users to visualize and organize large datasets effectively. Through features like drag-and-drop functionality and customizable dashboards, researchers can manage genomic data more intuitively, making it easier to track samples and results.

3. Customization Flexibility:

A well-designed GUI offers customization options that enable researchers to tailor their workflows according to specific experimental needs. Users can adjust settings or modify visual layouts to suit their preferences, enhancing the overall usability of the software.

4. Increased Accessibility:

GUIs make complex technologies more accessible to a wider range of users, including those who may not have a strong technical background. By lowering the barrier to entry for using NGS tools, GUIs democratize access to genomic research and analysis.

5. Facilitated Collaboration:

Collaborative features integrated into GUIs enable multiple users to work together seamlessly on genomic projects. Shared access to data visualizations and analysis tools fosters teamwork among researchers, enhancing productivity and innovation in precision medicine initiatives.

Introduction to Gradio Framework

Gradio is an open-source Python library designed to simplify the creation of user-friendly web interfaces for machine learning models, APIs, or any Python function. It allows developers to build interactive applications with minimal code, making it a popular choice among machine learning practitioners, educators, and data scientists.



Key Features of Gradio

- 1. Ease of Use:** Gradio provides a simple and intuitive way to create web interfaces. It supports a wide range of input and output components, such as textboxes, images, and audio, allowing developers to design interactive and responsive user experiences.
- 2. Core Classes:** Gradio offers foundational classes like `Interface` and `Blocks`. The `Interface` class is ideal for quick demos, connecting functions to input and output components with minimal setup. The `Blocks` class offers more customization options, allowing developers to build complex applications.
- 3. Components:** Gradio includes over 30 built-in components, including `Textbox`, `Image`, `Button`, and `Slider`. These components can be easily integrated into applications to enhance user interaction.
- 4. Hugging Face Integration:** Gradio is closely integrated with Hugging Face, a leading platform in the machine learning space. This integration allows users to leverage Hugging Face models and Spaces to host and share Gradio demos easily.
- 5. Sharing and Deployment:** Gradio apps can be shared via a public link or deployed on platforms like Hugging Face Spaces, making them accessible to a broader audience.

CHAPTER 4: REVIEW OF LITERATURE

REVIEW OF LITERATURE

NEXT – GENERATION SEQUENCING (NGS) TOOLS

Bioinformatics tools are essential software applications and algorithms designed to analyze, interpret, and visualize biological data, particularly genomic and proteomic information. Their importance lies in their ability to handle vast amounts of data generated by high-throughput sequencing technologies, enabling researchers to make sense of complex biological systems and contribute to advancements in fields such as genomics, transcriptomics, and proteomics.

Categories of Bioinformatics Tools

Bioinformatics tools can be classified based on their functionality, which includes:

- 1. Data Acquisition:** Tools like the SRA Toolkit facilitate the retrieval of sequencing data from public repositories. They allow researchers to download and manage large datasets efficiently, which is crucial for subsequent analyses.
- 2. Quality Control:** Tools such as FastQC are vital for assessing the quality of sequencing data. They provide metrics on various aspects, including sequence quality scores and GC content, helping researchers identify potential issues before proceeding with further analysis.
- 3. Pre-processing:** Fastp is an example of a tool that offers comprehensive pre-processing capabilities, including adapter trimming and quality filtering. This step is critical for ensuring that only high-quality reads are used in downstream analyses.

These tools collectively enhance the efficiency and accuracy of bioinformatics analyses, allowing researchers to focus on interpreting results rather than managing data intricacies. The growing number of specialized tools reflects the increasing complexity of biological questions being addressed in modern research.

I. SRA TOOLKIT

The **SRA Toolkit** is a collection of command-line tools designed to facilitate access to and management of data stored in the Sequence Read Archive (SRA), a public repository for high-throughput sequencing data. This toolkit serves as a vital resource for researchers looking to retrieve and analyze genomic data efficiently.



The SRA Toolkit allows users to download, convert, and manipulate sequencing data from the SRA. It supports various types of sequencing data, making it an essential tool for bioinformaticians and researchers in genomics. The toolkit is particularly advantageous for those who require large

datasets for their analyses, as it streamlines the process of accessing these resources.

Key Features

1. **Data Access:** The SRA Toolkit provides seamless access to a vast array of datasets available in the SRA. Users can easily search for specific studies or datasets using identifiers such as the Study ID or Run ID.
2. **Downloading Methods:** The toolkit offers multiple downloading options, including direct downloads and the ability to download specific formats like FASTQ. This flexibility allows users to choose the method that best suits their computational environment.
3. **Formats Supported:** The SRA Toolkit supports various file formats, including SRA, FASTQ, and BAM files. This versatility enables researchers to work with different types of data without needing extensive conversions.

Applications

1. **Genomic Research:** Researchers utilize the toolkit to access large-scale genomic datasets for studies on population genetics, evolutionary biology, and disease association.
2. **Comparative Genomics:** It facilitates comparative analyses by allowing easy retrieval of datasets from different organisms or conditions.
3. **Method Development:** Bioinformaticians often use the toolkit to test new algorithms or methods on existing datasets, contributing to advancements in genomic analysis techniques.

In summary, the SRA Toolkit is an indispensable resource that enhances the accessibility and usability of genomic data, supporting a wide range of research applications in genomics and beyond.

II. FASTQC

FastQC is a widely used bioinformatics tool designed for quality control of high-throughput sequencing data. Its primary purpose is to provide a comprehensive overview of the quality of sequencing reads, helping researchers identify potential issues that may affect downstream analyses.

FastQC serves as an essential first step in the analysis pipeline, offering insights into the quality of raw sequencing data. By evaluating various aspects of the data, it enables researchers to make informed decisions about subsequent processing steps, such as filtering or trimming reads to enhance data quality.

Key Metrics Analyzed

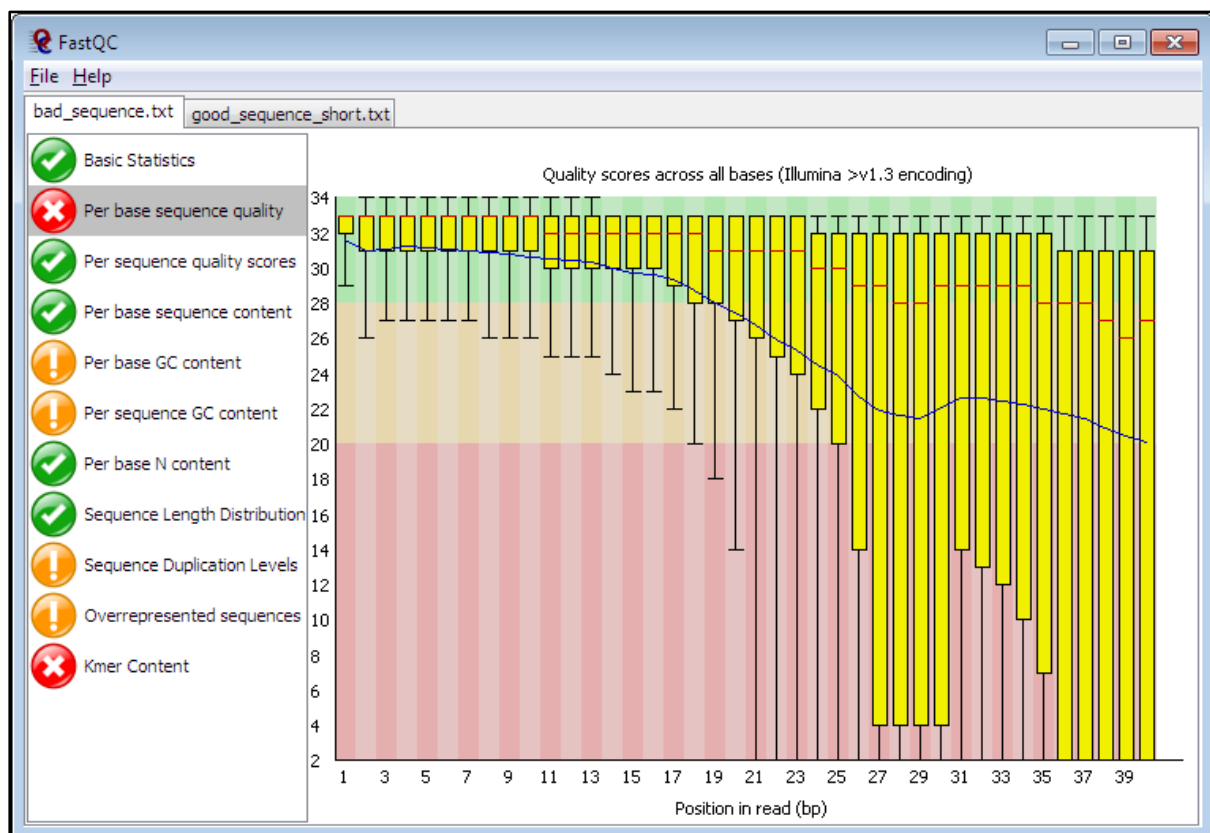


Figure 1: Sample FASTQC Report

1. Basic Statistics:

This section provides fundamental information about the input FASTQ file, including:

- File Name:** The name of the analyzed file.
- Quality Score Encoding:** Indicates the encoding method used for quality scores.
- Total Number of Reads:** The total count of sequences in the file.
- Read Length:** The length of the reads, which is crucial for downstream analyses.
- GC Content:** The percentage of guanine (G) and cytosine (C) bases, which can provide insights into the composition of the sequenced material.

2. Per Base Sequence Quality:

This module presents a box-and-whisker plot showing quality scores at each base position across all reads. Key features include:

- a. **Mean Quality Score:** Represented by a blue line, indicating average quality.
- b. **Median Quality Score:** Shown as a red line within a yellow box, providing insight into the central tendency of quality scores.
- c. **Quality Score Ranges:** The background color indicates quality levels (green for good, orange for acceptable, red for poor). A common observation is that quality tends to decrease towards the end of reads due to signal decay²³⁴.

3. Per Sequence Quality Scores:

This plot displays the distribution of average quality scores across all sequences. It helps identify if a significant portion of sequences has low quality, which could indicate issues such as poor imaging or contamination.

4. Per Sequence GC Content:

This module shows the distribution of GC content across all sequences. A normal distribution is expected in whole genome shotgun sequencing, with deviations potentially indicating contamination or over-representation of certain sequences.

5. Per Base N Content:

This metric indicates the percentage of bases called as 'N' (unknown) at each position. A high percentage at any position suggests problems during sequencing, such as instrument errors or insufficient signal.

6. Sequence Duplication Levels:

This analysis assesses how many times each unique sequence appears in the dataset. High levels of duplication can indicate issues with library preparation or PCR amplification bias.

7. Overrepresented Sequences:

FastQC identifies any sequences that are present at unusually high frequencies, which may suggest contamination or bias in library preparation. This module is crucial for detecting potential issues before further analysis.

8. Adapter Content:

This section checks for the presence of adapter sequences that may not have been completely removed during library preparation. High levels of adapter contamination can affect downstream applications and analyses.

Interpretation of Results

Interpreting FastQC output involves examining the provided visualizations and metrics:

1. **Quality Score Graphs:** A consistent high-quality score across all bases indicates good sequencing quality. A significant drop towards the end of reads may suggest the need for trimming.
2. **GC Content Plot:** A GC content that deviates significantly from expected values may indicate contamination or bias in library preparation.
3. **Adapter Content:** If adapter sequences are detected, it signals that additional trimming is necessary before proceeding with further analysis.

Interpretation of Flags

Each module in FastQC is assigned a flag:

1. **Pass (Green Tick):** Indicates no issues detected.
2. **Warn (Orange Exclamation Mark):** Suggests caution; potential issues should be investigated further.
3. **Fail (Red Cross):** Indicates significant problems that need addressing before proceeding with analysis.

In summary, FastQC is an invaluable tool for ensuring high-quality sequencing data. By analyzing critical metrics and providing clear visualizations, it empowers researchers to make informed decisions regarding data processing and analysis.

III. FASTP

Fastp is an innovative and efficient pre-processing tool designed for high-throughput sequencing data. It streamlines the initial steps of data analysis by providing a comprehensive suite of functionalities that enhance data quality and usability.

Fastp serves as a versatile pre-processing tool that integrates multiple functions into a single application, allowing researchers to handle raw sequencing data efficiently. It is particularly suitable for applications requiring rapid processing of large datasets, making it a popular choice in genomic studies.

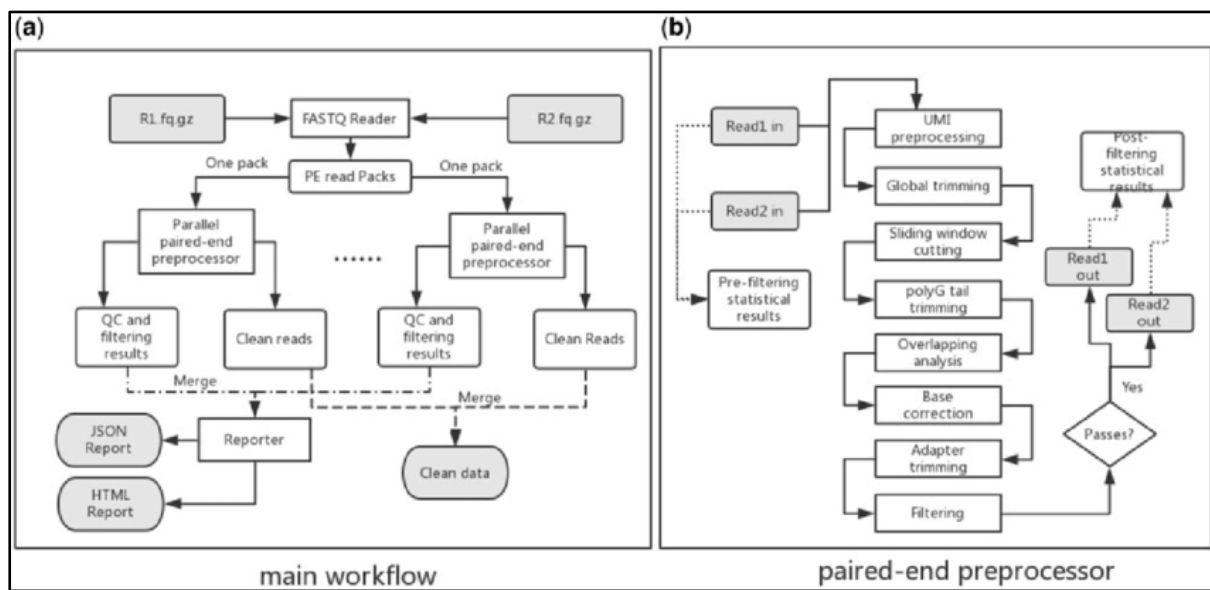


Figure 2: Workflow for the processing of files using the Fastp tool

Features

Fastp offers several key features that distinguish it from other pre-processing tools:

- 1. Quality Filtering:** Fastp employs dynamic quality filtering, which allows users to set thresholds for base quality scores. This ensures that only high-quality reads are retained for downstream analysis, effectively reducing noise and improving overall data integrity.
- 2. Adapter Trimming:** One of Fastp's standout features is its ability to detect and trim adapter sequences automatically. This is crucial because untrimmed adapters can lead to erroneous results in subsequent analyses. Fastp identifies adapter sequences based on user-defined or built-in patterns, ensuring clean reads.
- 3. Read Correction:** Fastp includes a read correction feature that addresses errors in sequencing data. By leveraging overlapping reads, it can correct low-quality bases, enhancing the reliability of the data before further analysis.

Performance Comparison

When compared to other tools like FastQC, Fastp stands out due to its integrated approach. While FastQC primarily focuses on quality assessment rather than pre-processing, Fastp combines quality control with essential pre-processing functions. This dual capability allows researchers to streamline their workflow significantly. In terms of speed and efficiency, Fastp has been shown to outperform many traditional tools by processing large datasets more rapidly without compromising accuracy. Additionally, Fastp generates comprehensive reports that provide insights into the quality of the processed data, similar to what FastQC offers but with the added advantage of immediate corrective actions. In summary, Fastp is a powerful pre-processing tool that enhances the quality of sequencing data through its advanced features, making it an essential component in modern bioinformatics workflows. Its combination of quality filtering, adapter trimming, and read correction positions it as a superior choice for researchers seeking efficient and reliable data preparation methods.

CHAPTER 5: METHODOLOGY

METHODOLOGY

The methodology for developing an integrated Next-Generation Sequencing (NGS) workflow with a user-friendly Graphical User Interface (GUI) encompasses several systematic phases. These phases ensure a comprehensive approach from problem identification to the final release of the software. The methodology is structured under the following key headers:

STEP 1: Problem Statement and Literature Review

1. Problem Identification:

The project initiates with the identification of critical challenges in existing NGS workflows, particularly focusing on the complexities associated with data analysis, interpretation, and the lack of intuitive user interfaces that hinder accessibility for clinicians and researchers in precision medicine.

2. Comprehensive Literature Review:

A thorough review of current literature is conducted to examine existing NGS workflows, precision medicine applications, and GUI design principles. This review aims to identify best practices, uncover gaps in current methodologies, and establish a foundation for developing an enhanced, user-centric NGS pipeline. Key areas of focus include:

- a. NGS Workflow Efficacy:** Evaluation of existing pipelines, their strengths, and limitations in handling large-scale sequencing data.
- b. GUI Design Principles:** Investigation of user interface designs that enhance usability, accessibility, and functionality for end-users in biomedical research settings.



STEP 2: Pipeline Designing and Testing

1. Workflow Design:

The NGS workflow is meticulously mapped out to include all essential steps from raw data acquisition to final reporting. The designed workflow encompasses:

- a. Data Acquisition:** Downloading raw NGS data (reads in FASTQ format) using tools like SRA Toolkit.
- b. Quality Control:** Performing quality filtering and adapter trimming using FastQC and Fastp to ensure data integrity.

2. Pipeline Testing:

Each step of the pipeline undergoes rigorous testing using sample datasets to validate functionality, ensure data integrity, and optimize performance. Automated scripts and shell scripting are employed to facilitate task automation, manage file operations, and execute command-line tools efficiently.



STEP 3: Backend Development and Integration of NGS Tools

1. Programming Language Selection:

The backend development leverages a combination of programming languages to harness their respective strengths:

- a. **Python:** Utilized for its extensive libraries that facilitate NGS data analysis and GUI development.
- b. **Shell Scripting:** Used for automating various tasks within the NGS workflow, enhancing efficiency in file management and tool execution.
- c. **Additional Languages:** May be incorporated as needed to support specialized data analysis and processing requirements.

2. Integration of Bioinformatics Tools:

Existing bioinformatics tools and algorithms are seamlessly integrated into the backend to support comprehensive data analysis, visualization, and interpretation of NGS results. This integration ensures that users can perform end-to-end analyses within a unified platform without the need for external tool dependencies.

NGS Pipeline Implementation:

Downloading Raw NGS Data:

Utilizes SRA Toolkit to download raw sequencing reads in FASTQ format from public repositories.



Quality Filtering and Adapter Trimming:

Employs FastQC for initial quality assessment and Fastp for filtering low-quality reads and trimming adapter sequences to ensure high-quality input data.



STEP 4: GUI/Frontend or Software Development

1. Prototype Development:

An initial prototype of the GUI is developed using suitable development tools and Gradio frameworks that support graphical interface design. The prototype serves as a foundational model to visualize workflow steps and gather preliminary user feedback.

2. User Interface Design:

The GUI is designed following best practices in user interface design to ensure it is intuitive, responsive, and accessible. Key features include:

- a. **Workflow Visualization:** Clear mapping of NGS workflow steps, allowing users to navigate through data acquisition, analysis, and reporting seamlessly.
- b. **Interactive Elements:** Incorporation of interactive components such as buttons, progress bars, and data visualization tools to enhance user engagement.
- c. **Customization Options:** Providing users with the ability to configure analysis parameters and customize reports based on specific research needs.



STEP 5: Validation and Optimization

1. Performance Validation:

The developed GUI and integrated NGS pipeline are subjected to extensive testing using real NGS datasets. This phase assesses the accuracy, reliability, and efficiency of the workflow. Key performance indicators include:

- a. **Data Processing Speed:** Measuring the time taken for each pipeline step to ensure timely analysis.
- b. **Accuracy of Results:** Verifying the correctness of variant calling and data interpretations against known benchmarks.
- c. **System Reliability:** Ensuring the software operates consistently without crashes or data loss.

2. Optimization Strategies:

Based on validation results, algorithms and workflow processes are optimized to enhance performance. Optimization efforts focus on:

- a. **Algorithm Refinement:** Improving the efficiency and accuracy of data processing algorithms.
- b. **Resource Management:** Enhancing the software's ability to handle large datasets without compromising performance.
- c. **User Experience Enhancements:** Streamlining processes to reduce user effort and improve overall satisfaction.



STEP 6: Documentation and Release

1. Comprehensive Documentation:

Detailed documentation is developed to cover all aspects of the project, including step-by-step guides on how to use the software, perform analyses, and interpret results.

2. Final Release Preparation:

The software undergoes final quality assurance checks to ensure all components function as intended. Packaging includes the executable software, documentation, and necessary dependencies to facilitate easy installation and usage by end-users.

3. Release and Deployment:

The finalized software is released to the target audience through appropriate distribution channels. Ongoing support and updates are planned to address any emerging issues, incorporate user feedback, and implement future enhancements.

CHAPTER 6:

CODE

CODE

```
import os
import sys
import subprocess
import requests
from tqdm import tqdm
import shutil
import time
import tarfile
import gradio as gr
from dotenv import load_dotenv

def update_path(path_var, path):
    with open(".env", "r") as f:
        lines = f.readlines()
    for i, line in enumerate(lines):
        if path is not None and path_var in line:
            lines[i] = f"{path_var}='{path}'\n"
    with open(".env", "w") as f:
        f.writelines(lines)

def add_path(path_var, path):
    with open(".env", "a") as f:
        if path is not None:
            f.write(f"{path_var}='{path}'\n")

def set_paths(path_var, temppath):
    if not os.path.isfile('.env'):
        try:
            with open('.env', "w") as f:
                f.write("")
        except Exception as e:
            print(f"Error creating .env file: {e}")
            sys.exit(1)
    try:
        with open(".env", "r") as f:
            content = f.read()
        if path_var in content:
            update_path(path_var, temppath)
        else:
            add_path(path_var, temppath)
    except Exception as e:
        print("Error updating path:", e)
        sys.exit(1)

def run_command_out(cmd, dir=None):
    cmd_list = cmd.split()
    if dir is not None:
```

```

        if not os.path.isabs(cmd_list[0]):
            cmd_list[0] = os.path.join(dir, cmd_list[0])
    try:
        output = subprocess.check_output(cmd_list)
        return output.decode("utf-8")
    except subprocess.CalledProcessError as e:
        print("Error running command:", e.output.decode())
        return cmd

def run_command(cmd, dir=None):
    cmd_list = cmd.split()
    if dir is not None:
        if not os.path.isabs(cmd_list[0]):
            cmd_list[0] = os.path.join(dir, cmd_list[0])
    try:
        subprocess.run(cmd_list, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
        return True
    except subprocess.CalledProcessError as e:
        return False

def download(url, fname, chunk_size=1024):
    headers = {
        'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537.3'
    }
    if os.path.isfile(fname):
        print(f"File '{fname}' already exists.")
    else:
        try:
            resp = requests.get(url, headers=headers, stream=True)
            resp.raise_for_status()
            total = int(resp.headers.get('content-length', 0))
            with open(fname, 'wb') as file, tqdm(desc=fname, total=total,
unit='iB', unit_scale=True, unit_divisor=1024) as bar:
                for data in resp.iter_content(chunk_size=chunk_size):
                    size = file.write(data)
                    bar.update(size)
            print(f"Downloaded '{fname}' successfully.")
        except requests.exceptions.RequestException as e:
            print(f"Download error: {e}")
            return None
    return fname

def install_sra():
    print("SRA Toolkit not found. Downloading now...")
    fname = "sratoolkit.tar.gz"

```

```

    file = download(url="https://ftp-
trace.ncbi.nlm.nih.gov/sra/sdk/3.1.1/sratoolkit.3.1.1-ubuntu64.tar.gz",
fname=fname)
    if not file:
        print("Download failed.")
        return
    print("SRA Toolkit downloaded.")
    with tarfile.open(file, "r:gz") as tar:
        tar.extractall()
    print("SRA Toolkit extracted.")
    extracted_dir = "sratoolkit.3.1.1-ubuntu64"
    os.remove(fname)
    if os.path.exists(extracted_dir):
        os.rename(extracted_dir, "sratoolkit")
        print("SRA Toolkit installed.")
    else:
        print(f"Error: '{extracted_dir}' not found after extraction.")

load_dotenv()
app_dir = os.getenv("APP_DIR", "applications")
data_dir = os.getenv("DATA_DIR", "data")
sratoolkit_path = os.getenv("SRATOOLKIT_PATH")
fastqc_path = os.getenv("FASTQC_PATH")
fastp_path = os.getenv("FASTP_PATH")

def setup_sudo(sudo_password):
    if sudo_password is None:
        return "SUDO_PASSWORD is not set in .env file."
    try:
        commands = ["sudo -S apt-get update"]
        output_log = ""
        for command in commands:
            proc = subprocess.Popen(command.split(), stdin=subprocess.PIPE,
stdout=subprocess.PIPE, stderr=subprocess.PIPE)
            output, error = proc.communicate(input=(sudo_password +
'\n').encode())
            output_log += output.decode("utf-8")
            if error:
                output_log += "Error: " + error.decode("utf-8") + "\n"
        return output_log
    except Exception as e:
        return f"Error setting up sudo: {e}"

def setup_deps(sudo=True):
    try:
        if sudo:
            run_command_out("sudo apt-get install -y wget")
            run_command_out("sudo apt-get install -y python3-pip")

```

```

        run_command_out("sudo apt-get install -y default-jdk")
        run_command_out("sudo apt-get install -y default-jre")
        run_command_out("sudo apt-get install -y unzip")
        run_command_out("sudo apt-get install -y build-essential")
        run_command_out("sudo apt-get install -y bwa")
        run_command_out("gcc --version")
        run_command_out("sudo apt-get install -y zlib1g-dev")
        return "Dependencies installed with sudo."
    else:
        run_command_out("apt-get install -y wget")
        run_command_out("apt-get install -y python3-pip")
        run_command_out("apt-get install -y default-jdk")
        run_command_out("apt-get install -y default-jre")
        run_command_out("apt-get install -y unzip")
        run_command_out("apt-get install -y build-essential")
        run_command_out("apt-get install -y bwa")
        run_command_out("gcc --version")
        run_command_out("apt-get install -y zlib1g-dev")
        return "Dependencies installed without sudo."
except Exception as e:
    return f"Error installing dependencies: {e}"

def setup_sra(app_dir):
    sra_path_local = f"{app_dir}/sratoolkit/bin"
    try:
        run_command_out("vdb-dump --help", dir=sra_path_local)
    except Exception:
        run_command_out(f"wget -O {app_dir}/sratoolkit.tar.gz https://ftp-trace.ncbi.nlm.nih.gov/sra/sdk/3.1.1/sratoolkit.3.1.1-ubuntu64.tar.gz")
        run_command_out(f"tar -xzf {app_dir}/sratoolkit.tar.gz -C {app_dir}")
        run_command_out(f"mv {app_dir}/sratoolkit.3.1.1-ubuntu64 {app_dir}/sratoolkit")
        run_command_out(f"rm {app_dir}/sratoolkit.tar.gz")
    return f"{app_dir}/sratoolkit/bin"

def setup_fastqc(app_dir):
    fastqc_path_local = f"{app_dir}/fastqc"
    try:
        run_command_out("fastqc --help", dir=fastqc_path_local)
    except Exception:
        run_command_out(f"wget -O {app_dir}/fastqc.zip https://www.bioinformatics.babraham.ac.uk/projects/fastqc/fastqc_v0.12.1.zip")
        run_command_out(f"unzip {app_dir}/fastqc.zip -d {app_dir}")
        run_command_out(f"rm {app_dir}/fastqc.zip")
        run_command_out(f"mv {app_dir}/FastQC {app_dir}/fastqc-temp")
        run_command_out(f"mv {app_dir}/fastqc-temp {app_dir}/fastqc")
    return f"{app_dir}/fastqc"

```

```

def setup_fastp(app_dir):
    fastp_path_local = f"{app_dir}/fastp"
    os.makedirs(fastp_path_local, exist_ok=True)
    try:
        run_command_out("fastp --help", dir=fastp_path_local)
    except Exception:
        run_command_out(f"wget -O {fastp_path_local}/fastp
http://opengene.org/fastp/fastp")
        run_command_out(f"chmod a+x {fastp_path_local}/fastp")
    return fastp_path_local

def run_setup(sudo_password):
    os.makedirs(app_dir, exist_ok=True)
    os.makedirs(data_dir, exist_ok=True)
    log = ""
    log += setup_sudo(sudo_password) + "\n"
    log += setup_deps(sudo=True) + "\n"
    sra_p = setup_sra(app_dir)
    set_paths("SRAToolkit_PATH", sra_p)
    global sratoolkit_path
    sratoolkit_path = sra_p
    log += f"Set SRA Toolkit path: {sra_p}\n"
    fastqc_p = setup_fastqc(app_dir)
    set_paths("FASTQC_PATH", fastqc_p)
    log += f"Set FastQC path: {fastqc_p}\n"
    fastp_p = setup_fastp(app_dir)
    set_paths("FASTP_PATH", fastp_p)
    log += f"Set Fastp path: {fastp_p}\n"
    log += "Setup completed successfully."
    return log

def sra_cmd_for_info(accession: str):
    toolkit_path = os.getenv("SRAToolkit_PATH") or f"{app_dir}/sratoolkit/bin"
    vdb_dump_path = os.path.join(toolkit_path, "vdb-dump")
    return f"{vdb_dump_path} {accession} --info"

def sra_cmd_to_download(accession: str,
                        alignment_filter: bool,
                        compressed: bool,
                        skip_technical: bool,
                        remove_adapter: bool,
                        spot_group: bool,
                        alignment_filter_type: str = None,
                        min_reads: int = None,
                        max_reads: int = None,
                        ar_specific: str = None,
                        ar_start: int = None,
                        ar_end: int = None,

```

```

        member: str = None):
    toolkit_path = os.getenv("SRAToolkit_PATH") or f"{app_dir}/sratoolkit/bin"
    fastq_dump_path = os.path.join(toolkit_path, "fastq-dump")
    cmd_list = [fastq_dump_path, accession, '--split-3', '--outdir',
f'{data_dir}/{accession}']
    if compressed:
        cmd_list.append('--gzip')
    if alignment_filter:
        if alignment_filter_type == "split-spot":
            if '--split-3' in cmd_list: cmd_list.remove('--split-3')
            cmd_list.append('--split-spot')
        elif alignment_filter_type == "aligned":
            if '--split-3' in cmd_list: cmd_list.remove('--split-3')
            cmd_list.append('--aligned')
        elif alignment_filter_type == "unaligned":
            if '--split-3' in cmd_list: cmd_list.remove('--split-3')
            cmd_list.append('--unaligned')
        elif alignment_filter_type == "aligned-region":
            if '--split-3' in cmd_list: cmd_list.remove('--split-3')
            cmd_list.append('--aligned-region')
            cmd_list.append(f'{ar_specific}:{ar_start}-{ar_end}')
        elif alignment_filter_type == "matepair-distance":
            if '--split-3' in cmd_list: cmd_list.remove('--split-3')
            cmd_list.append('--matepair-distance')
            cmd_list.append(f'{ar_start}-{ar_end}')
    if skip_technical:
        cmd_list.append('--skip-technical')
    if min_reads:
        cmd_list.extend(['--minSpotId', str(min_reads)])
    if max_reads:
        cmd_list.extend(['--maxSpotId', str(max_reads)])
    if remove_adapter:
        cmd_list.append('--clip')
    if spot_group:
        cmd_list.extend(['--spot-group', member])
    return ' '.join(cmd_list)

def list_files_in_directory(directory):
    try:
        return [os.path.join(directory, f) for f in os.listdir(directory)
                if os.path.isfile(os.path.join(directory, f))]
    except Exception as e:
        return []

def compress_dir(directory, zip_filename):
    import zipfile
    with zipfile.ZipFile(zip_filename, mode='w') as zip_file:
        for root, dirs, files in os.walk(directory):

```

```

        for file in files:
            file_path = os.path.join(root, file)
            zip_file.write(file_path, arcname=os.path.relpath(file_path,
directory))
        return zip_filename

def sra_download_main(accession: str,
                      alignment_filter_type: str,
                      alignment_filter: bool,
                      compressed: bool,
                      skip_technical: bool,
                      remove_adapter: bool,
                      spot_group: bool,
                      min_reads: int = None,
                      max_reads: int = None,
                      ar_specific: str = None,
                      ar_start: int = None,
                      ar_end: int = None,
                      member: str = None):
    try:
        shutil.rmtree(data_dir)
    except FileNotFoundError:
        print("Data folder not found; proceeding.")
    except OSError as e:
        print("Error removing data folder:", e)
    sra_info_cmd = sra_cmd_for_info(accession)
    try:
        run_command_out(sra_info_cmd)
    except Exception as e:
        try:
            install_sra()
        except Exception as e:
            print("Error during SRA installation:", e)
    work_dir = os.path.join(data_dir, accession)
    os.makedirs(work_dir, mode=0o777, exist_ok=True)

    os.environ["WORKING_DIR"] = work_dir
    get_data_cmd = sra_cmd_for_info(accession)
    download_cmd = sra_cmd_to_download(accession, alignment_filter,
compressed,
                                skip_technical, remove_adapter,
spot_group,
                                alignment_filter_type, min_reads,
max_reads,
                                ar_specific, ar_start, ar_end, member)
    log = f"Fetching data with: {get_data_cmd}\n"
    data_info = run_command_out(get_data_cmd)
    log += f>Data info: {data_info}\n"

```

```

log += f"Downloading Fastq files with: {download_cmd}\n"
download_status = run_command_out(download_cmd)
log += f"Download status: {download_status}\n"
zip_file = compress_dir(work_dir, os.path.join(data_dir,
f"{accession}.zip"))
print(f>Data downloaded successfully for {accession}")
return data_info, download_status, zip_file

def fastqc_main(uploaded_files):
    working_dir = os.getenv("WORKING_DIR")
    if not working_dir:
        return "Working directory not set. Run the SRA download step first.",
[]

    for file_obj in uploaded_files:
        dest = os.path.join(working_dir, os.path.basename(file_obj.name))
        shutil.copy(file_obj.name, dest)
        print(f"Copied {file_obj.name} to {dest}")
    output_dir = os.path.join(working_dir, 'fastqc_reports')
    os.makedirs(output_dir, exist_ok=True)

    for f in os.listdir(working_dir):
        file_path = os.path.join(working_dir, f)
        if any(ext in f for ext in [".fastq", ".fasta", ".fa", ".sam",
".bam"]):
            cmd = f"fastqc {file_path} --outdir {output_dir}"
            print(f"Running FastQC command: {cmd}")
            try:
                run_command_out(cmd, dir=fastqc_path)
            except Exception as e:
                print(f"Error running FastQC on {file_path}: {e}")

    reports = [os.path.join(output_dir, f) for f in os.listdir(output_dir)
                if f.endswith(".html")]
    if not reports:
        print("No FastQC reports were generated.")
    else:
        print("Generated FastQC reports:", reports)
    return "FastQC processing completed.", reports

def fastp_main(uploaded_files):
    working_dir = os.getenv("WORKING_DIR")
    if not working_dir:
        return "Working directory not set. Run the SRA download step first.",
[]

    if len(uploaded_files) < 2:
        return "Please upload two FASTQ files (forward and reverse).", []
    file1, file2 = uploaded_files[0], uploaded_files[1]

```

```

dest1 = os.path.join(working_dir, os.path.basename(file1.name))
dest2 = os.path.join(working_dir, os.path.basename(file2.name))
shutil.copy(file1.name, dest1)
shutil.copy(file2.name, dest2)
output_file_1 = os.path.join(working_dir,
os.path.basename(dest1).replace('.fastq.gz', '_trimmed.fastq.gz'))
output_file_2 = os.path.join(working_dir,
os.path.basename(dest2).replace('.fastq.gz', '_trimmed.fastq.gz'))
output_dir = os.path.join(working_dir, 'fastp_reports')
os.makedirs(output_dir, exist_ok=True)
command = (f"{fastp_path}/fastp"
           f" -i {dest1}"
           f" -o {output_file_1}"
           f" -I {dest2}"
           f" -O {output_file_2}"
           f" --detect_adapter_for_pe"
           f" -f 10 -g -l 50"
           f" -c -h {output_dir}/fastp_report.html"
           f" -w 10")

try:
    run_command_out(command)
    trimmed_dir = os.path.join(working_dir, 'trimmed')
    os.makedirs(trimmed_dir, exist_ok=True)
    shutil.move(output_file_1, os.path.join(trimmed_dir,
os.path.basename(output_file_1)))
    shutil.move(output_file_2, os.path.join(trimmed_dir,
os.path.basename(output_file_2)))
    trimmed_files = [os.path.join(trimmed_dir,
os.path.basename(output_file_1)),
                    os.path.join(trimmed_dir,
os.path.basename(output_file_2))]
    return "Fastp processing completed.", trimmed_files
except Exception as e:
    return f"Error running Fastp: {e}", []

def enable_spot_group(esg):
    return gr.update(visible=True) if esg else gr.update(visible=False)

def enable_reads_count(erc):
    return (gr.update(visible=True), gr.update(visible=True)) if erc else
(gr.update(visible=False), gr.update(visible=False))

def enable_alignment_filter(eaf):
    return gr.update(visible=True) if eaf else gr.update(visible=False)

def enable_alignment_filter_type(eaft):
    if eaft == "aligned-region":

```

```

        return gr.update(visible=True), gr.update(visible=True),
gr.update(visible=True)
        elif eaft == "matepair-distance":
            return gr.update(visible=False), gr.update(visible=True),
gr.update(visible=True)
        else:
            return gr.update(visible=False), gr.update(visible=False),
gr.update(visible=False)

def get_fastq(accession, alignment_filter_type, alignment_filter, compression,
              skip_technical, remove_adapter, spot_group, min_reads,
max_reads,
              ar_specific, ar_start, ar_end, member):
    print("Received parameters:")
    print("accession:", accession)
    print("alignment_filter_type:", alignment_filter_type)
    print("alignment_filter:", alignment_filter)
    print("compression:", compression)
    print("skip_technical:", skip_technical)
    print("remove_adapter:", remove_adapter)
    print("spot_group:", spot_group)
    print("min_reads:", min_reads)
    print("max_reads:", max_reads)
    print("ar_specific:", ar_specific)
    print("ar_start:", ar_start)
    print("ar_end:", ar_end)
    print("member:", member)
    data_info, download_status, file_zip = sra_download_main(accession,
alignment_filter_type, alignment_filter,
                                                              compression,
skip_technical, remove_adapter,
                                                              spot_group,
min_reads, max_reads,
                                                              ar_specific,
ar_start, ar_end, member)
    return data_info, download_status, file_zip

with gr.Blocks(title="NGS Sequence Quality Check") as demo:
    gr.Markdown("# NGS Sequence Quality Check")
    gr.Markdown("Streamline NGS workflows for quality checking using the SRA
Toolkit, FastQC, and Fastp.")

    with gr.Tab("Setup Environment"):
        gr.Markdown("## Setup Environment & Install Dependencies")
        sudo_input = gr.Textbox(label="Enter Sudo Password", type="password",
placeholder="Sudo Password")
        setup_button = gr.Button("Run Setup")
        setup_output = gr.Textbox(label="Setup Log", interactive=False)

```

```

        setup_button.click(fn=run_setup, inputs=sudo_input,
outputs=setup_output)

    with gr.Tab("SRA Downloader") as webui:
        gr.Markdown("## SRA Toolkit")
        with gr.Row():
            with gr.Column():
                accession = gr.Textbox(label="Enter SRA Number",
placeholder="ERR11468775", interactive=True)
            with gr.Column():
                compression = gr.Checkbox(label="Compress", value=True)
                skip_technical = gr.Checkbox(label="Skip Technical
details", value=False)
                remove_adapter = gr.Checkbox(label="Remove Adapter",
value=False)
                spot_group = gr.Checkbox(label="Enable Spot Grouping",
value=False)
                member = gr.Textbox(label="Member", placeholder="Enter
Member", interactive=True, visible=False)
            with gr.Column():
                apply_min_max_reads = gr.Checkbox(label="Enable Min & Max
Reads Count", value=False)
                min_reads = gr.Number(label="Minimum Reads", visible=False)
                max_reads = gr.Number(label="Maximum Reads", visible=False)
            with gr.Column():
                alignment_filter = gr.Checkbox(label="Apply Alignment
Filters", value=False)
                alignment_filter_type = gr.Radio(label="Alignment Filter
Type",
                                                    choices=["split-spot",
"aligned", "unaligned", "aligned-region", "matepair-distance"],
                                                    visible=False)
            with gr.Column():
                ar_specific = gr.Textbox(label="Aligned Region Specific",
placeholder="Enter Specific Region", interactive=True, visible=False)
                ar_start = gr.Number(label="Aligned Region Start",
visible=False)
                ar_end = gr.Number(label="Aligned Region End", visible=False)
                apply_min_max_reads.change(fn=enable_reads_count,
inputs=apply_min_max_reads, outputs=[min_reads, max_reads])
                alignment_filter.change(fn=enable_alignment_filter,
inputs=alignment_filter, outputs=[alignment_filter_type])
                alignment_filter_type.change(fn=enable_alignment_filter_type,
inputs=alignment_filter_type, outputs=[ar_specific, ar_start, ar_end])
                spot_group.change(fn=enable_spot_group, inputs=spot_group,
outputs=[member])
                btn2 = gr.Button(value="Download SRA Data")
        with gr.Row():

```

```

        data = gr.Textbox(label="Data", placeholder="Data", lines=10,
interactive=True)
        download_status = gr.Textbox(label="Download Status",
placeholder="Download Status", lines=5, interactive=True)
        files = gr.File(label="Download your File")
        btn2.click(fn=get_fastq,
                    inputs=[accession, alignment_filter_type, alignment_filter,
compression, skip_technical,
                            remove_adapter, spot_group, min_reads, max_reads,
ar_specific, ar_start, ar_end, member],
                    outputs=[data, download_status, files])

    with gr.Tab("FastQC"):
        gr.Markdown("## FastQC Tool")
        gr.Markdown("Upload unzipped FASTQ/FASTA files (forward and reverse
reads).")
        fastqc_files_input = gr.File(label="Upload FASTQ/FASTA Files",
file_count="multiple")
        fastqc_run_button = gr.Button("Run FastQC")
        fastqc_log = gr.Textbox(label="FastQC Log", interactive=False)
        fastqc_reports = gr.File(label="FastQC Reports",
file_count="multiple")
        fastqc_run_button.click(fn=fastqc_main,
                                inputs=fastqc_files_input,
                                outputs=[fastqc_log, fastqc_reports])

    with gr.Tab("Fastp"):
        gr.Markdown("## Fastp Tool")
        gr.Markdown("Upload FASTQ files for adapter trimming (Forward and
Reverse).")
        fastp_files_input = gr.File(label="Upload FASTQ Files",
file_count="multiple")
        fastp_run_button = gr.Button("Run Fastp")
        fastp_log = gr.Textbox(label="Fastp Log", interactive=False)
        fastp_reports = gr.File(label="Trimmed FASTQ Files",
file_count="multiple")
        fastp_run_button.click(fn=fastp_main,
                                inputs=fastp_files_input,
                                outputs=[fastp_log, fastp_reports])

    try:
        webui.queue(default_concurrency_limit=25)
    except Exception as e:
        print(f"Queue error: {e}")

demo.launch()

```

CHAPTER 7: RESULTS

RESULTS

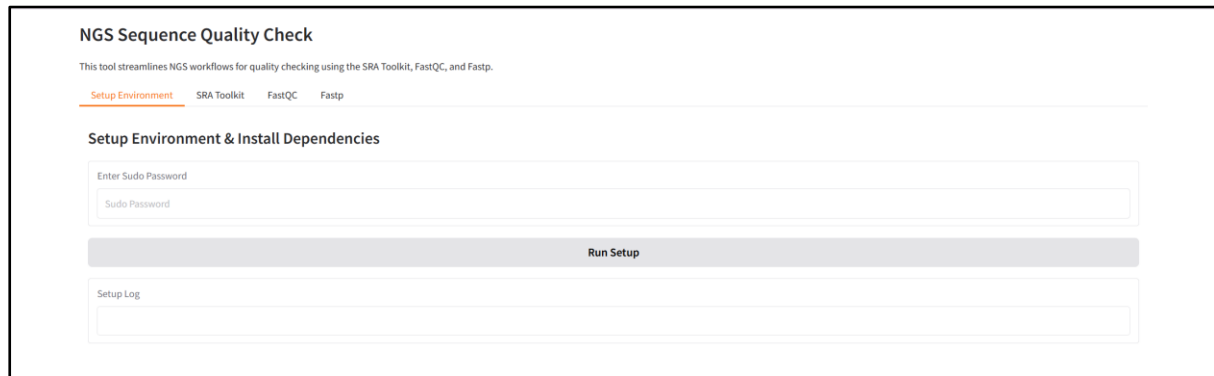


Figure 3: Graphical User Interface – Gradio Framework – ‘Setup Environment’ tab

```
prarthi@LAPTOP-FV19MUSI:/mnt/c/Users/prarthi kothari/OneDrive/Desktop/NGSQualityCheck$ /bin/python3 "/mnt/c/Users/prarthi kothari/OneDrive/Desktop/NGSQualityCheck/app.py"
* Running on local URL:  http://127.0.0.1:7860

To create a public link, set `share=True` in `launch()`.
--2025-03-10 00:12:30-- https://ftp-trace.ncbi.nlm.nih.gov/sra/sdk/3.1.1/sratoolkit.3.1.1-ubuntu64.tar.gz
Resolving ftp-trace.ncbi.nlm.nih.gov (ftp-trace.ncbi.nlm.nih.gov)... 130.14.250.12, 130.14.250.11, 2607:f220:41e:250::11, ...
Connecting to ftp-trace.ncbi.nlm.nih.gov (ftp-trace.ncbi.nlm.nih.gov)|130.14.250.12|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 93532905 (89M) [application/x-gzip]
Saving to: 'applications/sratoolkit.tar.gz'

applications/sratoolkit.tar.gz      100%[=====>] 89.20M  1.33MB/s   in 77s

2025-03-10 00:13:54 (1.16 MB/s) - 'applications/sratoolkit.tar.gz' saved [93532905/93532905]

--2025-03-10 00:13:58-- https://www.bioinformatics.babraham.ac.uk/projects/fastqc/fastqc_v0.12.1.zip
Resolving www.bioinformatics.babraham.ac.uk (www.bioinformatics.babraham.ac.uk)... 149.155.133.4
Connecting to www.bioinformatics.babraham.ac.uk (www.bioinformatics.babraham.ac.uk)|149.155.133.4|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 11709692 (11M) [application/zip]
Saving to: 'applications/fastqc.zip'

applications/fastqc.zip            100%[=====>] 11.17M  1.32MB/s   in 8.2s

2025-03-10 00:14:07 (1.37 MB/s) - 'applications/fastqc.zip' saved [11709692/11709692]

--2025-03-10 00:14:13-- http://opengene.org/fastp/fastp
Resolving opengene.org (opengene.org)... 8.210.133.117
Connecting to opengene.org (opengene.org)|8.210.133.117|:80... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://opengene.org/fastp/fastp [following]
--2025-03-10 00:14:13-- https://opengene.org/fastp/fastp
Connecting to opengene.org (opengene.org)|8.210.133.117|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 9528432 (9.1M) [application/octet-stream]
Saving to: 'applications/fastp/fastp'

applications/fastp/fastp          100%[=====>] 9.09M  1016KB/s   in 7.6s

2025-03-10 00:14:24 (1.20 MB/s) - 'applications/fastp/fastp' saved [9528432/9528432]
```

Figure 4: Setting up the environment, downloading and installation of required tools

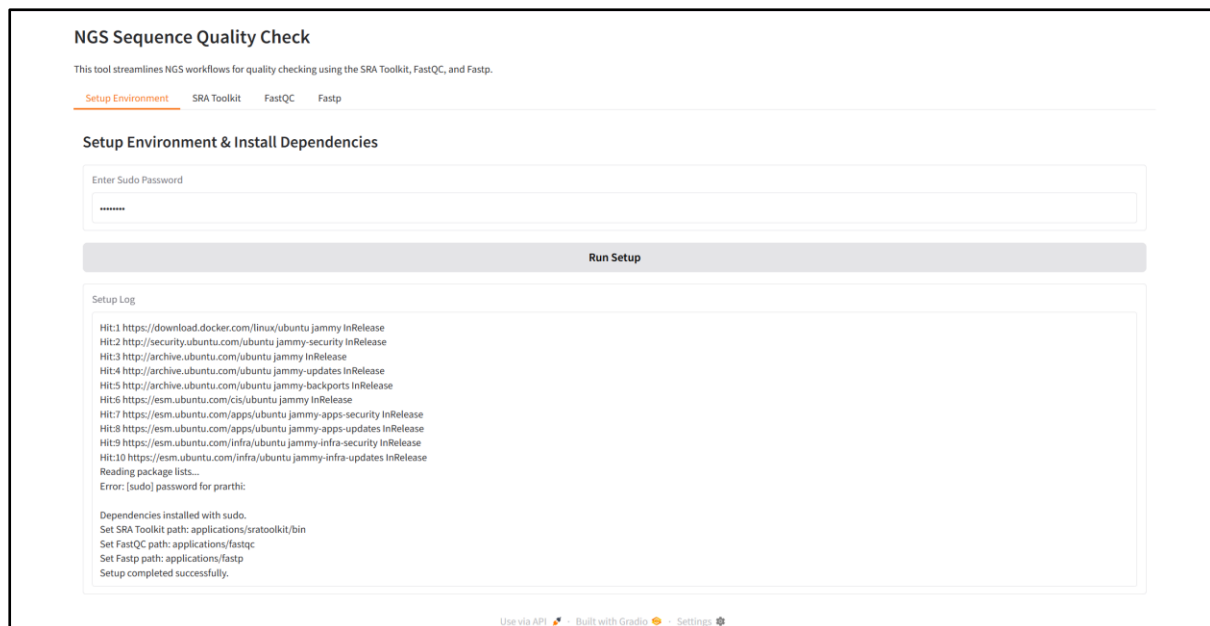


Figure 5: Setting up the environment and installing dependencies complete

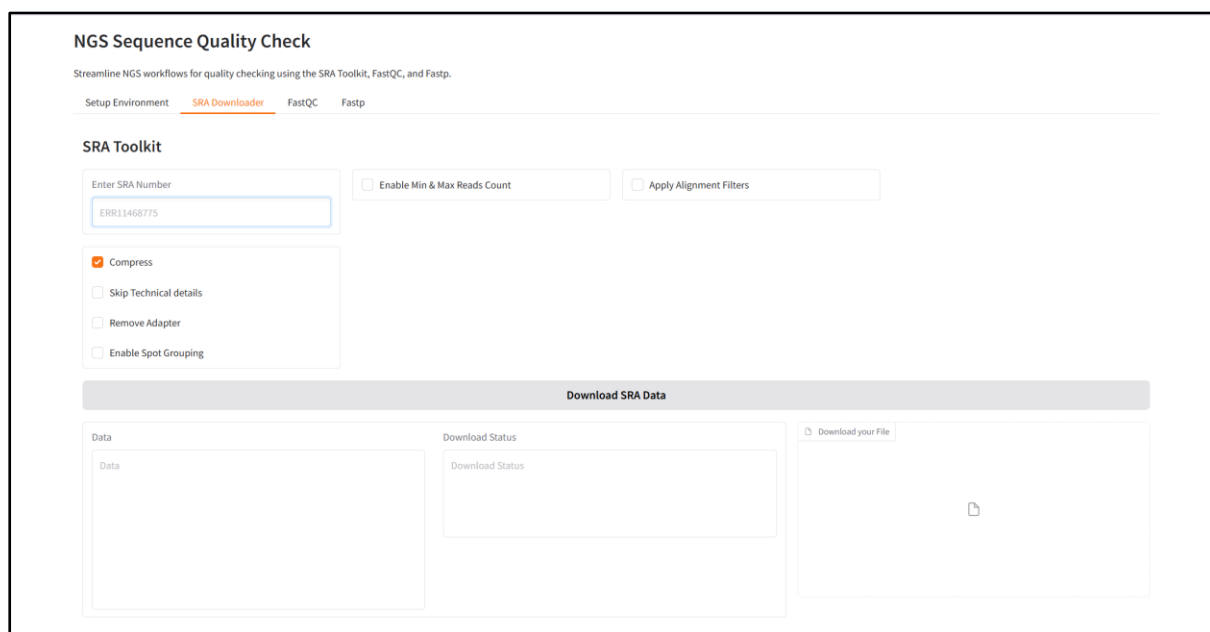


Figure 6: Graphical User Interface – Gradio Framework – ‘SRA Downloader’ tab

NGS Sequence Quality Check
Streamline NGS workflows for quality checking using the SRA Toolkit, FastQC, and Fastp.

Setup Environment **SRA Downloader** FastQC Fastp

SRA Toolkit

Enter SRA Number
ERR11468775

☒ Compress
☐ Skip Technical details
☐ Remove Adapter
☐ Enable Spot Grouping

☒ Enable Min & Max Reads Count
Minimum Reads
0
Maximum Reads
100

☒ Apply Alignment Filters
Alignment Filter Type
☐ split-spot ☐ aligned ☐ unaligned
☐ aligned-region ☐ matepair-distance

Download SRA Data

Data
Data

Download Status
Download Status

Download your File

Figure 7: ‘SRA Downloader’ tab – Inputting all the details and required parameters for retrieving reads data using the SRA Toolkit

```
Received parameters:
accession: ERR11468775
alignment_filter_type: None
alignment_filter: False
compression: True
skip_technical: False
remove_adapter: False
spot_group: False
min_reads: 0
max_reads: 100
ar_specific:
ar_start: 0
ar_end: 0
member:
Data downloaded successfully for ERR11468775
```

Figure 8: ‘SRA Downloader’ tab – Input received and processing at the backend

Setup Environment **SRA Downloader** FastQC Fastp

SRA Toolkit

Enter SRA Number
ERR11468775

☒ Compress
☐ Skip Technical details
☐ Remove Adapter
☐ Enable Spot Grouping

☒ Enable Min & Max Reads Count
 Minimum Reads: 0
 Maximum Reads: 100

☒ Apply Alignment Filters
 Alignment Filter Type:
☐ split-spot ☐ aligned ☐ unaligned
☐ aligned-region ☐ matepair-distance

Download SRA Data

Data

```

acc : ERR11468775
path : https://sra-pub-run-
odp.s3.amazonaws.com/sra/ERR11468775/ERR11468775
size : 49,663,706,774
type : Database
platf : SRA_PLATFORM_ILLUMINA
SEQ : 410,126,452
SCHEMA : NCBI:SRA/Illumina.db#2
TIME : 0x000000005de426e (02/28/2024 01:43)
FMT : shanq
FMTVER : 3.0.11
LDR : general-loader.3.0.8
LDRVER : 3.0.8
LDRDATE : Sep 11 2023 (9/11/2023 0:0)

```

Download Status

Read 100 spots for ERR11468775
 Written 100 spots for ERR11468775

Download your File
 ERR11468775.zip 21.7 KB

Figure 9: ‘SRA Downloader’ tab – Output received for the entered parameters

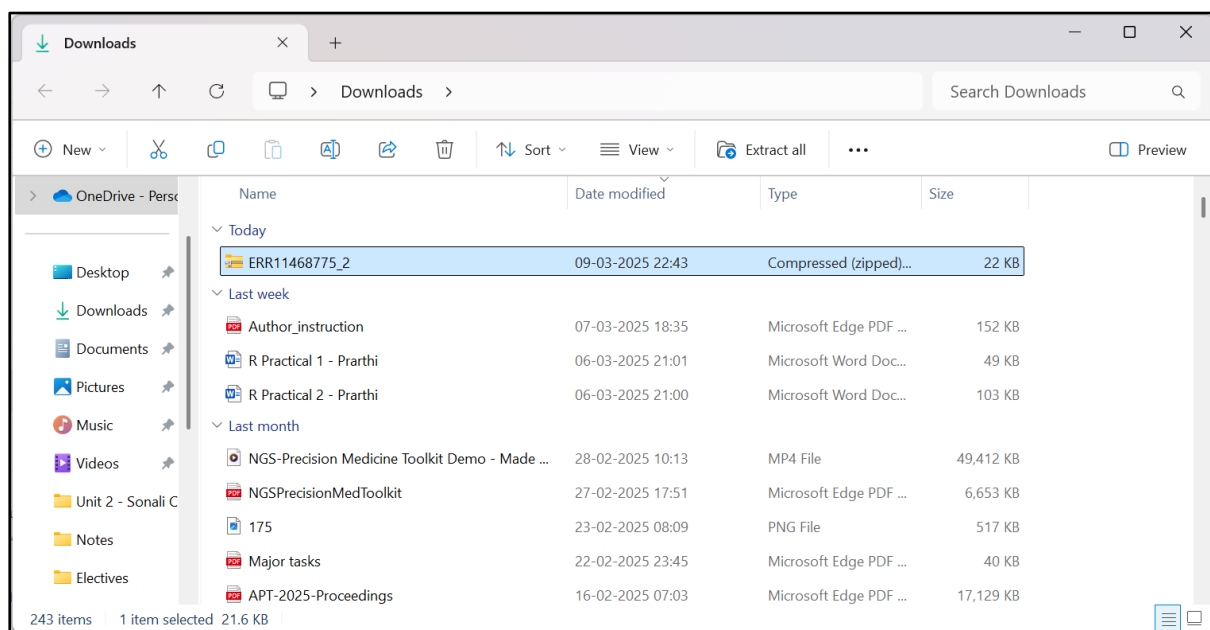


Figure 10: ‘SRA Downloader’ tab – Downloaded zip file of the sequences

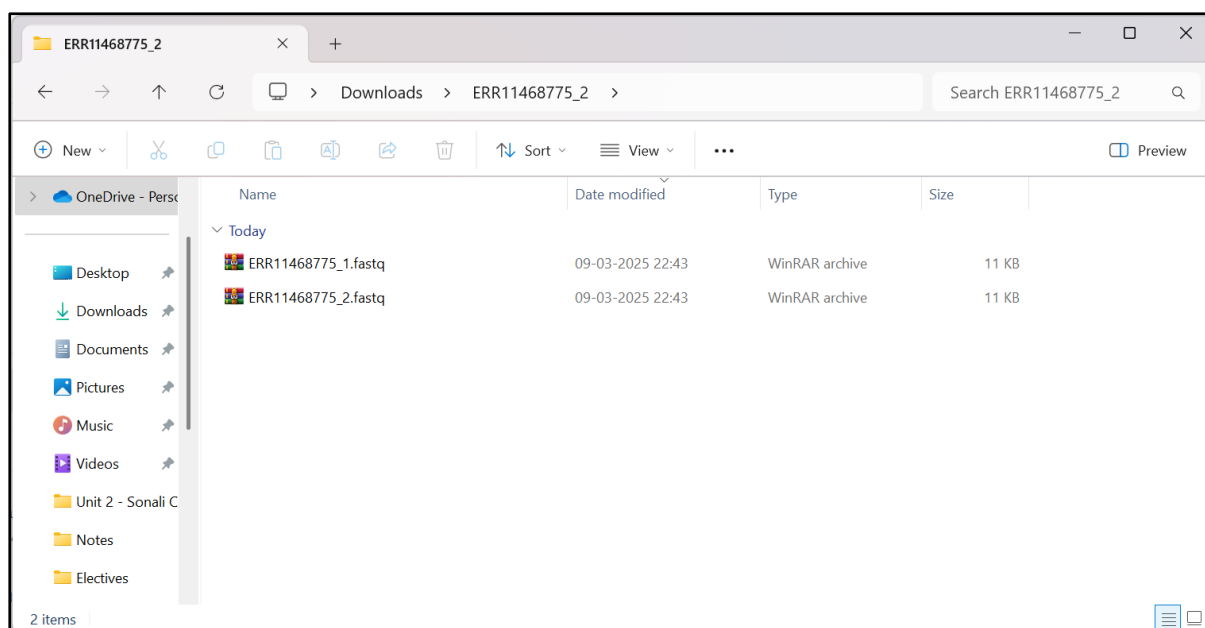


Figure 11: ‘SRA Downloader’ tab – Extracted output files

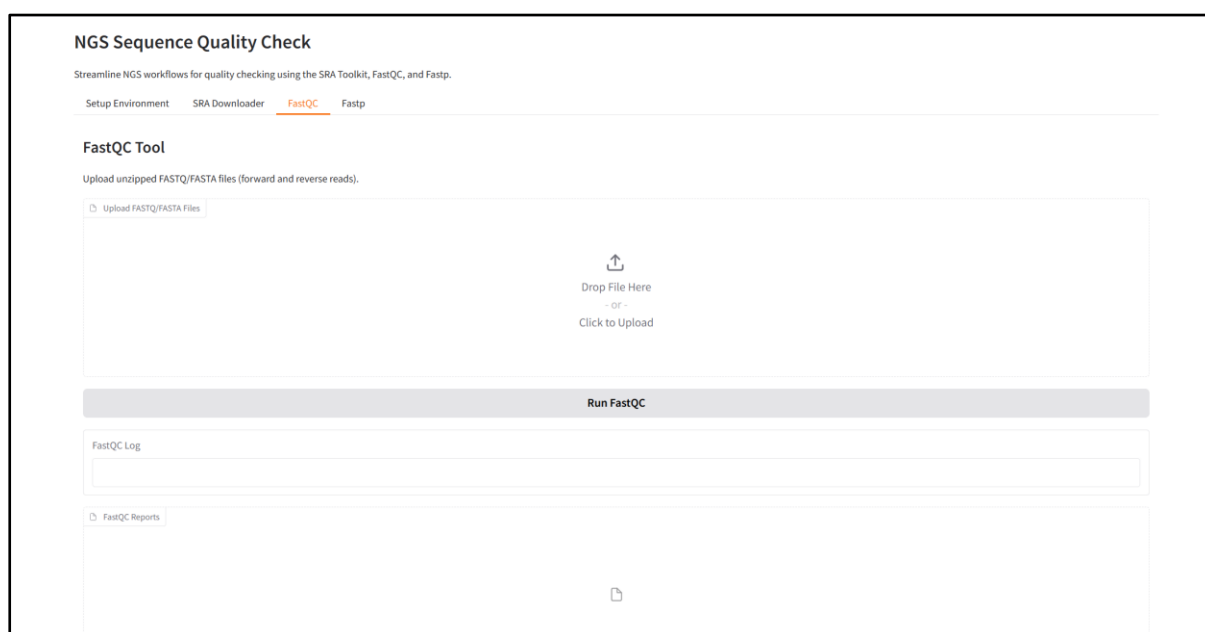


Figure 12: Graphical User Interface – Gradio Framework – ‘FastQC’ tab

NGS Sequence Quality Check

Streamline NGS workflows for quality checking using the SRA Toolkit, FastQC, and Fastp.

Setup Environment SRA Downloader **FastQC** Fastp

FastQC Tool

Upload unzipped FASTQ/FASTA files (forward and reverse reads).

Upload FASTQ/FASTA Files

ERR11468775_1.fastq	42.3 KB ↓ ×
ERR11468775_2.fastq	42.3 KB ↓ ×

Run FastQC

FastQC Log

FastQC Reports

Figure 13: ‘FastQC’ tab – Inputting all the details and required parameters for performing the quality check using FASTQC Tool

```
Copied /tmp/gradio/9af10038de8bf4b1aa5e553c8318f1ae52c7d5b5f97de547eab11af4eef697bc/ERR11468775_1.fastq to data/ERR11468775/ERR11468775_1.fastq
Copied /tmp/gradio/ab7a344be8d63cbcd7bf02cc9549e3e590882922976caa056584362e2b544934/ERR11468775_2.fastq to data/ERR11468775/ERR11468775_2.fastq
Running FastQC command: fastqc data/ERR11468775/ERR11468775_1.fastq --outdir data/ERR11468775/fastqc_reports
Started analysis of ERR11468775_1.fastq
Running FastQC command: fastqc data/ERR11468775/ERR11468775_1.fastq.gz --outdir data/ERR11468775/fastqc_reports
Started analysis of ERR11468775_1.fastq.gz
Running FastQC command: fastqc data/ERR11468775/ERR11468775_2.fastq --outdir data/ERR11468775/fastqc_reports
Started analysis of ERR11468775_2.fastq
Running FastQC command: fastqc data/ERR11468775/ERR11468775_2.fastq.gz --outdir data/ERR11468775/fastqc_reports
Started analysis of ERR11468775_2.fastq.gz
Generated FastQC reports: ['data/ERR11468775/fastqc_reports/ERR11468775_1_fastqc.html', 'data/ERR11468775/fastqc_reports/ERR11468775_2_fastqc.html']
```

Figure 14: ‘FastQC’ tab – Input received and processing at the backend

NGS Sequence Quality Check

Streamline NGS workflows for quality checking using the SRA Toolkit, FastQC, and Fastp.

Setup Environment SRA Downloader **FastQC** Fastp

FastQC Tool

Upload unzipped FASTQ/FASTA files (forward and reverse reads).

Upload FASTQ/FASTA Files

ERR11468775_1.fastq	42.3 KB ↓ ×
ERR11468775_2.fastq	42.3 KB ↓ ×

Run FastQC

FastQC Log

FastQC processing completed.

FastQC Reports

ERR11468775_1_fastqc.html	636.0 KB ↓ ×
ERR11468775_2_fastqc.html	658.7 KB ↓ ×

Use via API 🚀 · Built with Gradio 🍷 · Settings ⚙️

Figure 15: ‘FastQC’ tab – Output received for the entered parameters

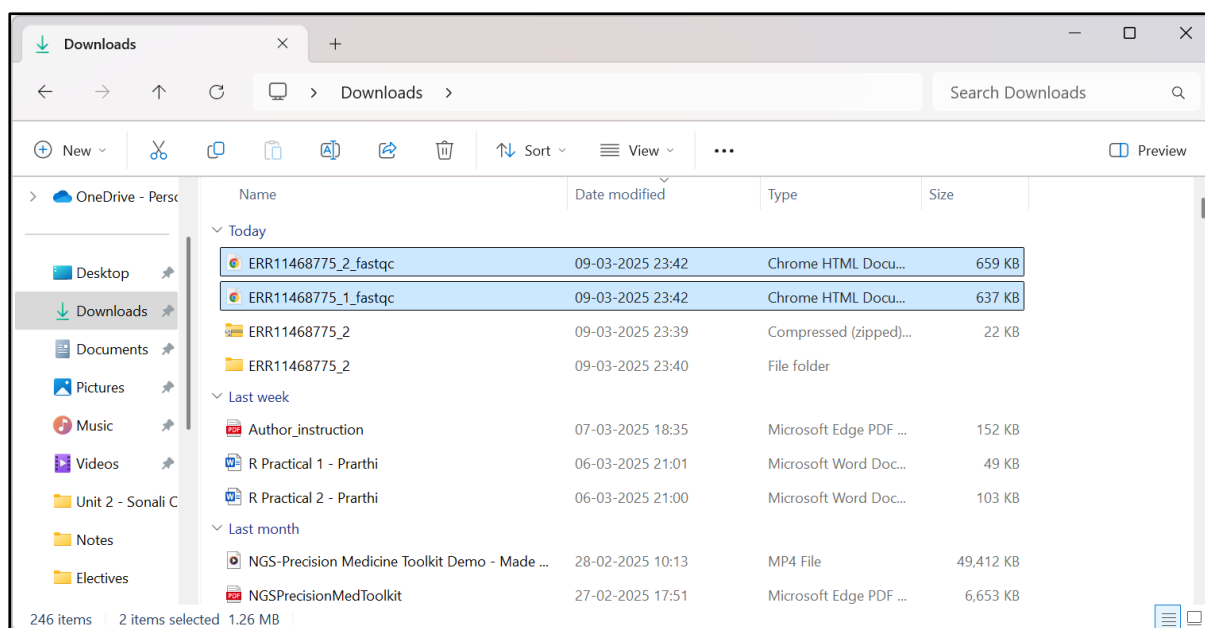


Figure 16: 'FastQC' tab – Downloaded FASTQC Reports for Quality Check

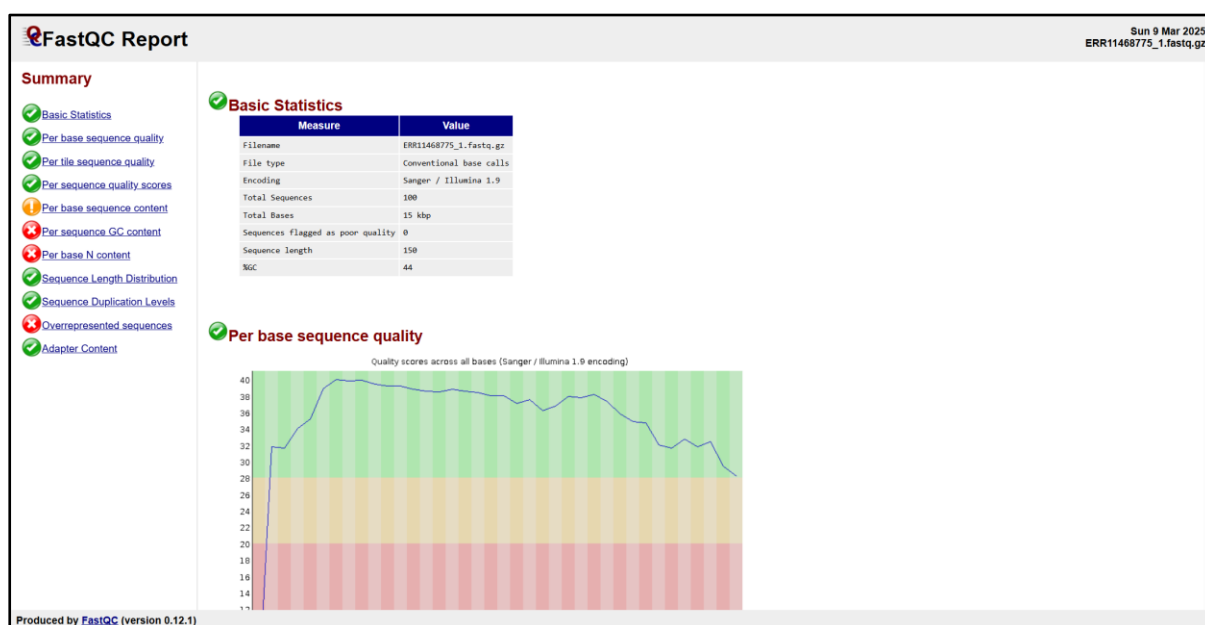


Figure 17: 'FastQC' tab – FASTQC Report for the forward read

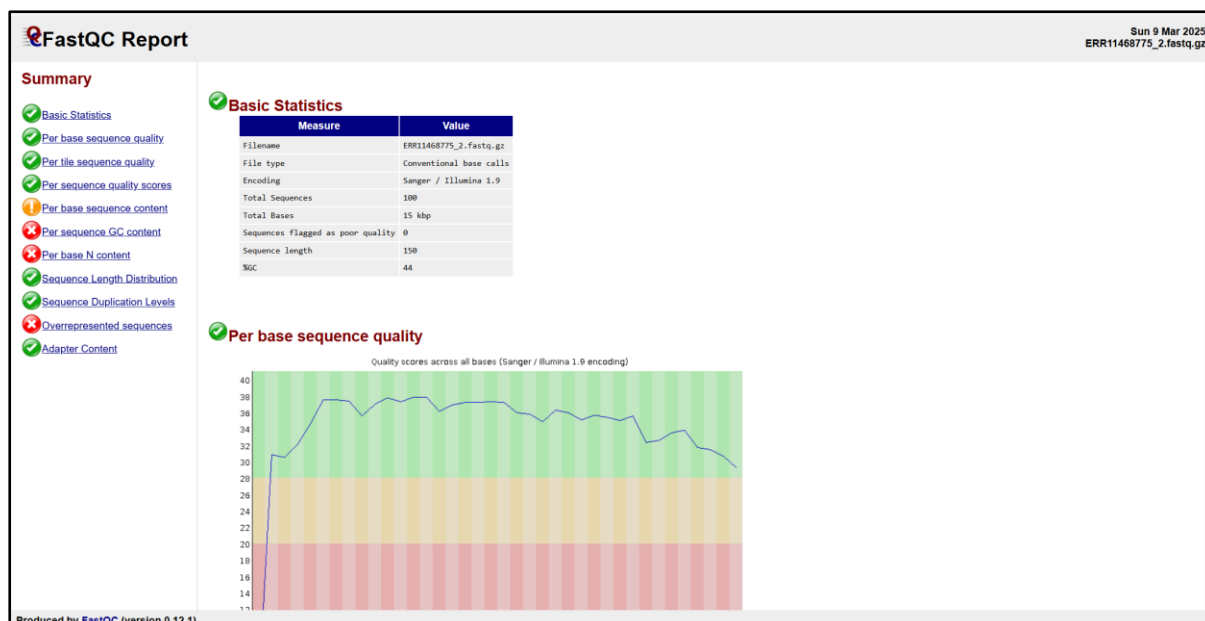


Figure 18: ‘FastQC’ tab – FASTQC Report for the reverse read

NGS Sequence Quality Check

Streamline NGS workflows for quality checking using the SRA Toolkit, FastQC, and Fastp.

Setup Environment SRA Downloader FastQC **Fastp**

Fastp Tool

Upload FASTQ files for adapter trimming (Forward and Reverse).

Upload FASTQ Files

Drop File Here
- OR -
Click to Upload

Run Fastp

Fastp Log

Trimmed FASTQ Files

Figure 19: Graphical User Interface – Gradio Framework – ‘Fastp’ tab

NGS Sequence Quality Check
Streamline NGS workflows for quality checking using the SRA Toolkit, FastQC, and Fastp.

Setup Environment SRA Downloader FastQC **Fastp**

Fastp Tool
Upload FASTQ files for adapter trimming (Forward and Reverse).

Upload FASTQ Files

ERR11468775_1.fastq	42.3 KB
ERR11468775_2.fastq	42.3 KB

Run Fastp

Fastp Log

Figure 20: ‘Fastp’ tab – Inputting all the details and required parameters for performing the quality check using Fastp Tool

NGS Sequence Quality Check
Streamline NGS workflows for quality checking using the SRA Toolkit, FastQC, and Fastp.

Setup Environment SRA Downloader FastQC **Fastp**

Fastp Tool
Upload FASTQ files for adapter trimming (Forward and Reverse).

Upload FASTQ Files

ERR11468775_1.fastq	42.3 KB
ERR11468775_2.fastq	42.3 KB

Run Fastp

Fastp Log

Fastp processing completed.

Figure 21: ‘Fastp’ tab – Output received for the entered parameters

fastp report	
Summary	
General	
fastp version:	0.23.4 (https://github.com/openGene/fastp)
sequencing:	paired end (150 cycles + 150 cycles)
mean length before filtering:	150bp, 150bp
mean length after filtering:	139bp, 139bp
duplication rate:	3.273200%
Insert size peak:	233
Before filtering	
total reads:	2.000000 M
total bases:	300.000000 M
Q20 bases:	285.264931 M (95.088310%)
Q30 bases:	267.838583 M (89.279528%)
GC content:	43.806530%
After filtering	
total reads:	1.919964 M
total bases:	267.648520 M
Q20 bases:	257.997977 M (96.394322%)
Q30 bases:	243.673713 M (91.042429%)
GC content:	43.683298%
Filtering result	
reads passed filters:	1.919964 M (95.998200%)
reads corrected:	115.245000 K (5.762250%)
bases corrected:	229.958000 K (0.076653%)
reads with low quality:	76.212000 K (3.810600%)
reads with too many N:	1.562000 K (0.078100%)

Figure 22: ‘Fastp’ tab – Fastp Report for the reads after processing and trimming the adapters using Fastp tool

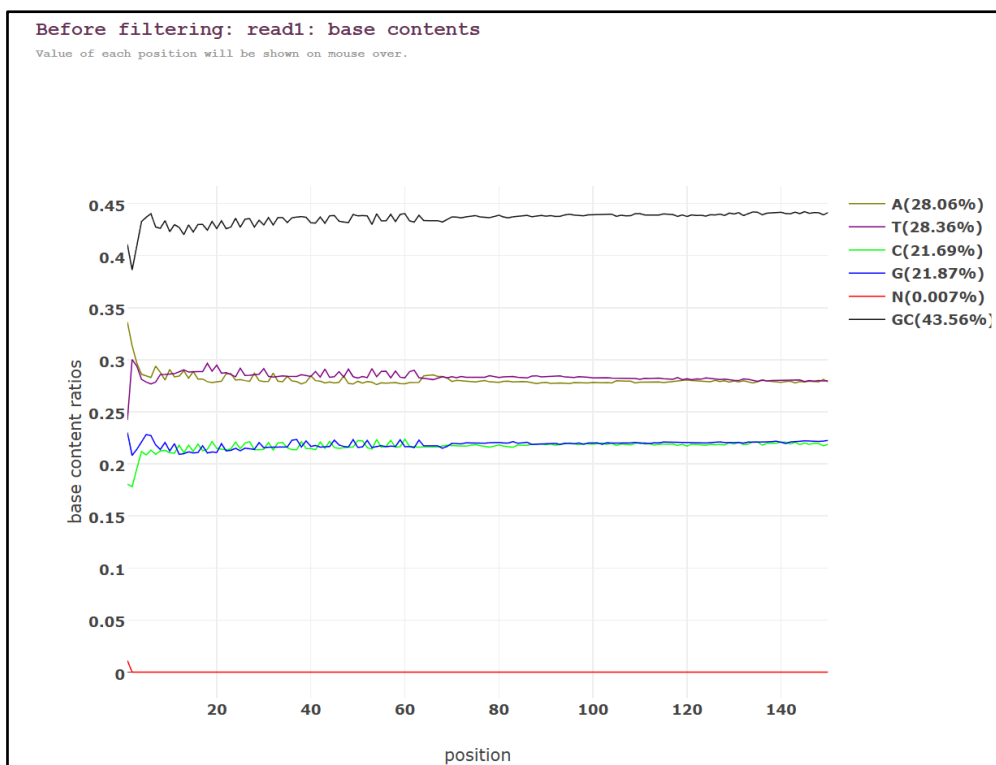


Figure 23: 'Fastp' tab – Fastp Report – Graph plot of base contents before filtering of the reads

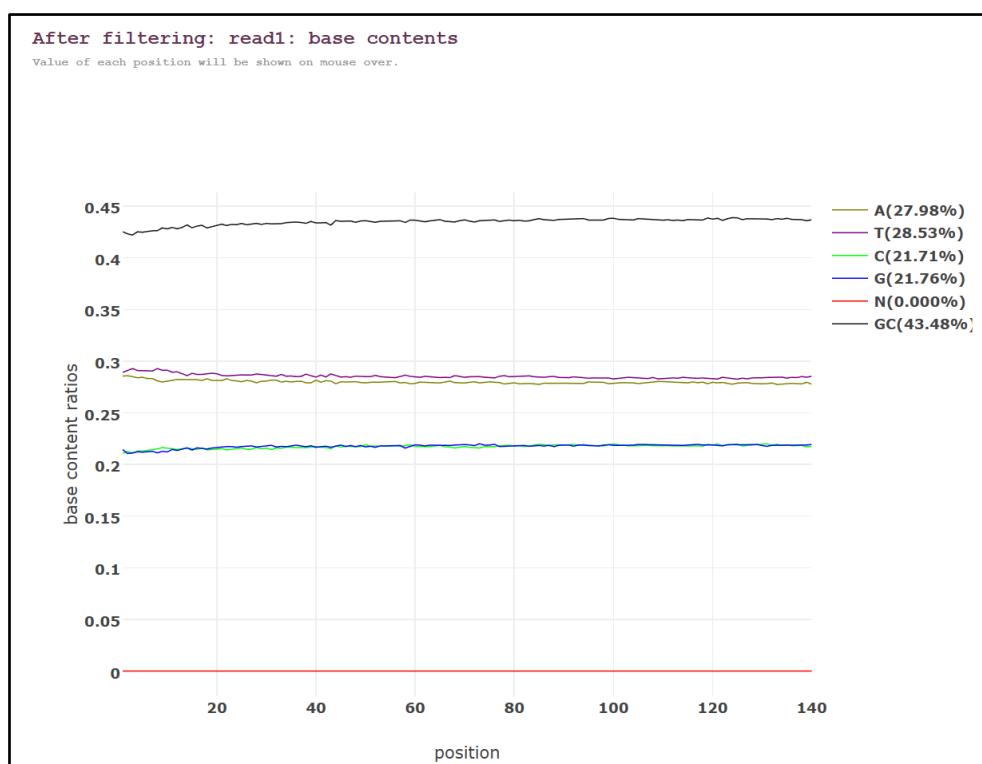


Figure 24: 'Fastp' tab – Fastp Report – Graph plot of base contents after filtering of the reads

CHAPTER 8: CONCLUSION

CONCLUSION

As NGS technology continues to evolve, several future directions and challenges emerge:

- 1. Integration with Clinical Practice:** The integration of NGS into routine clinical practice remains a challenge due to regulatory hurdles and the need for standardized protocols. Ensuring that genomic data is interpreted correctly and used effectively in clinical decision-making is crucial for maximizing its benefits.
- 2. Ethical Considerations:** The implications of genetic information raise ethical concerns regarding privacy, consent, and potential discrimination based on genetic predispositions. Addressing these issues will be essential as NGS becomes more prevalent in both research and healthcare settings.
- 3. Data Interpretation:** While data generation has become more efficient, interpreting the vast amounts of genomic data remains complex. Continued advancements in bioinformatics tools and algorithms are necessary to translate genomic findings into actionable insights effectively.

As technology continues to evolve, the role of GUIs in NGS workflows will likely expand further:

- 1. Integration with Artificial Intelligence:** Future GUIs may incorporate AI-driven analytics that assist users in interpreting complex genomic data more effectively.
- 2. Mobile Accessibility:** With the rise of mobile technology, developing mobile-friendly GUIs could allow researchers to access NGS tools on-the-go, increasing flexibility in data analysis.
- 3. Enhanced Visualization Techniques:** Continued advancements in visualization techniques will improve how genomic data is presented within GUIs, making it easier for users to understand intricate patterns and relationships within the data.

The detailed methodology outlines a structured approach to developing an integrated NGS workflow with a user-friendly GUI. By systematically addressing each phase—from problem identification and literature review to pipeline design, backend integration, validation, GUI development, and final release—the project ensures the creation of a robust, efficient, and accessible tool for precision medicine applications. Comprehensive documentation and iterative user feedback further guarantee that the final product meets the needs of its target users.

In summary, Graphical User Interfaces are vital for enhancing the usability and efficiency of NGS workflows. As technology progresses, GUIs will continue to play a crucial role in shaping the future of genomic analysis and personalized healthcare solutions through next-generation sequencing analysis.

CHAPTER 9: REFERENCES

REFERENCES

1. Andrews, S. (2010). FastQC: A Quality Control Tool for High Throughput Sequence Data [Online]. Available online at: <http://www.bioinformatics.babraham.ac.uk/projects/fastqc/>
 2. Appiah, Vincent. (2021). Download Fastq Data using SRA Toolkit. https://www.researchgate.net/publication/355651665_Download_Fastq_Data_using_SRA_Toolkit
 3. Babraham Bioinformatics - FastQC A Quality Control tool for High Throughput Sequence Data. (n.d.). <https://www.bioinformatics.babraham.ac.uk/projects/fastqc/>
 4. Chen, S., Zhou, Y., Chen, Y., & Gu, J. (2018). fastp: an ultra-fast all-in-one FASTQ preprocessor. *Bioinformatics* (Oxford, England), 34(17), i884–i890. <https://doi.org/10.1093/bioinformatics/bty560>
 5. Home - SRA - NCBI. (n.d.). <https://www.ncbi.nlm.nih.gov/sra>
 6. Khetani, M. P. R. (2018, September 5). Quality control: Assessing FASTQC results. Introduction to RNA-Seq using high-performance computing - ARCHIVED. https://hbctraining.github.io/Intro-to-rnaseq-hpc-salmon/lessons/qc_fastqc_assessment.html
 7. Morash, M., Mitchell, H., Beltran, H., Elemento, O., & Pathak, J. (2018). The Role of Next-Generation Sequencing in Precision Medicine: A Review of Outcomes in Oncology. *Journal of personalized medicine*, 8(3), 30. <https://doi.org/10.3390/jpm8030030>
 8. Ncbi. (n.d.). GitHub - ncbi/sra-tools: SRA Tools. GitHub. <https://github.com/ncbi/sra-tools>
 9. Schmid, S., Jochum, W., Padberg, B., Demmer, I., Mertz, K. D., Joerger, M., Britschgi, C., Matter, M. S., Rothschild, S. I., & Omlin, A. (2022). How to read a next-generation sequencing report-what oncologists need to know. *ESMO open*, 7(5), 100570. <https://doi.org/10.1016/j.esmoop.2022.100570>
-